

Smart Home Health Monitor

Final Presentation — CSC 494 IoT

Mausam Shrestha

Northern Kentucky University | Spring 2026

The Problem

Air purifiers lie about air quality.

Their sensors are **inside the device** — measuring air that has already been filtered.

The air quality at the purifier = **always clean**

The air quality where you breathe = **unknown**

Issue	Commercial Purifiers	This Project
Sensor location	Inside device	Breathing zone (3–6 ft)
Parameters monitored	PM2.5 only	CO2, PM2.5, VOC, Temp, Humidity
Works with any purifier	No — brand locked	Yes — via smart plug
Cost	\$200–400 per room	\$88 total

What I Built

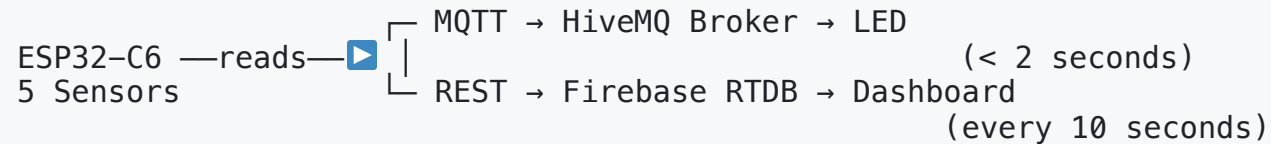
5 sensors on one XIAO ESP32-C6 — placed where you breathe

Component	Protocol	Status
MH-Z19C — CO2 (NDIR)	UART	✔ Working
DC01 — PM2.5	UART	⚠ Intermittent
MS1100 — VOC	Analog	✔ Working
AHT10 — Temp/Humidity	I2C	✔ Working
LCD1602 + LED actuator	I2C / GPIO	✔ Working

Data pushed to Firebase every **10 seconds** · MQTT actuator latency < **2 seconds** · All sensors tested and verified

How Everything Connects

Two separate communication paths — each optimized for its job:



Path 1 — Automation (MQTT)

ESP32 checks thresholds → publishes `ON` or `OFF` to a topic on HiveMQ's public broker → LED subscribes to the same topic and responds instantly. Fast, lightweight, no database involved.

Path 2 — Visibility (Firebase)

Every 10 seconds, ESP32 pushes all sensor readings to Firebase Realtime Database via REST API → Dashboard subscribes with `onValue` → updates instantly without polling.

Two paths because they solve different problems. MQTT is optimized for fast device-to-device control. Firebase is optimized for storing and streaming time-series data to any browser.

Automation + Dashboard

LED turns on automatically when any threshold is exceeded:

Sensor	Threshold	Action
CO2	> 1000 ppm	LED ON
PM2.5	> 35 µg/m³	LED ON
VOC	> 2500 raw ADC	LED ON

React + PWA dashboard (smart-home-health-monitor.web.app):

- Real-time sensor cards · 1h / 24h / 7d / All Data charts
- LED step timeline · Sensor health detection
- Sleep quality logger — built and deployed, ready to collect nightly scores
- Installable on phone — no App Store needed

System Verification

What was tested and confirmed working:

What was tested	Result
All 5 sensors reading simultaneously	✔ Verified
CO2 rising in a closed room	✔ Confirmed — readings climbed as expected
LED triggering when CO2 exceeded 1000 ppm	✔ Automation working
Firebase receiving readings every 10 seconds	✔ Verified in dashboard
Dashboard updating in real time	✔ Live at smart-home-health-monitor.web.app
PWA installable on phone	✔ Installed and working

The system is fully functional. Long-term continuous deployment — and using the sleep quality logger to build a correlation dataset — is the defined next step.

Learning with AI — Topic 1

Multi-Sensor Integration & Calibration

What I didn't know: I2C, UART, analog protocols, sensor calibration — none of it.

How AI helped:

- Explained I2C bus sharing — multiple devices, two wires, unique addresses
- Diagnosed my CO2 sensor showing constant 400 ppm → TX/RX wires were swapped
- Explained why NDIR CO2 (± 50 ppm) vs cheap estimated sensors (± 500 ppm) matters for health decisions
- Guided `millis()`-based non-blocking timing so all 5 sensors run simultaneously
- Taught moving average filtering to smooth noisy readings

What AI got wrong:

Told me to use GPIO21/22 for I2C — standard ESP32 pins. The XIAO ESP32-C6 uses **GPIO22/23**. Had to find the real answer in the Seeed Studio schematic. Lesson: AI knows common hardware. Uncommon boards need datasheet verification.

Learning with AI — Topic 2

Mobile App Development with React/PWA

My background: Several MERN stack projects — knew React well, never used Vite or Firebase.

How AI helped:

- Explained why Vite replaced CRA (< 1s startup vs 30s, actively maintained, CRA is deprecated)
- Taught the Observer Pattern — Firebase `onValue` *pushes* data instead of polling, the right model for a live sensor feed
- Guided `useMemo` for efficient client-side filtering across time ranges
- Explained PWA manifest — `display: standalone` makes it feel native, no App Store needed

What AI got wrong:

Recommended Firestore for the database. Firestore charges per document read — at 10-second pushes that adds up fast. Firebase **Realtime Database** is built for continuous time-series IoT data, free at this scale. Lesson: AI recommends the popular choice, not always the right one — always match the tool to the actual workload.

Results & What I'd Do Differently

Delivered:

-  5 sensors verified working simultaneously · \$88 total
-  Automated LED control via MQTT (< 2s latency)
-  Live PWA dashboard with sleep quality logger built and deployed
-  Full pipeline tested — sensor → threshold → MQTT → actuator → Firebase → dashboard

If I did it again:

- Deploy continuously for 4+ weeks to collect a real long-term dataset
- Log nightly sleep quality scores to build the CO2 vs sleep correlation
- Swap DC01 for PMS5003 — more reliable UART, laser scattering
- Use a real Tasmota smart plug — same MQTT message, zero code changes

Thank You

Any Questions?

Mausam Shrestha

shrestham2@mymail.nku.edu · CSC 494 IoT · Spring 2026

Live dashboard: smart-home-health-monitor.web.app

Code: github.com/shresthamausam07/smart-home-health-monitor

"The goal was to know what you're actually breathing — not what the purifier thinks you're breathing."